

Report - Sentiment Analysis

Advanced Machine Learning

Andres Robayo Chaves (2141627), Aryan Gupta (2152241), Bhavana Paladi (2136214),
Britta Boettcher (2147665) & Dhruva Etoor (2134652)

26th of October 2025

Contents

1	Introduction	2
2	Dataset	2
2.1	Dataset Creation	2
2.2	Data Preprocessing	2
2.3	Data Analysis	2
2.3.1	Descriptive Analysis	2
2.4	Bag-of-Words	2
2.4.1	Mutual Information Scores	3
3	Feature Engineering	3
3.1	Feature Representation	3
3.2	Feature Selection and Dimensionality Reduction	3
4	Training Procedure	4
5	Evaluation	4
6	Clustering and Topic Modeling	4
7	Discussion and Limitations	5
8	References	5
9	Appendix	6

1 Introduction

With the objective of improving our customer feedback analysis, we have compiled this report including different methods of text representation, feature selection, and modeling methods. Additionally, we also looked at clustering and topic modeling approaches to reveal principal themes in customer reviews. Ultimately, the methods proven as most useful, are meant to be used on reviews on our own e-commerce platform. For the sake of analysis, however, we will be utilizing a dataset containing Amazon Movies and TV reviews. The final model achieved an accuracy of approximately 82.6 %, indicating accurate sentiment tracking.

2 Dataset

2.1 Dataset Creation

The Amazon Movies and TV dataset contains 102,024 customer reviews. Each review has some vital information including **ratings**, **review text**, and **timestamps**. To analyse the customer feedback, we transformed this data into a binary classification problem. The models will classify each review as either positive or negative. We achieved this by creating a target variable called **high_rating** where ratings ≥ 4 are classified as positive **1** whereas ratings < 4 are classified as negative **0**.

The initial analysis of the data showed that approximately 81% of reviews are positive and only 19% are negative. As this imbalance would significantly impact model performance, we address this later through stratified sampling during cross-validation and using balanced evaluation metrics such as **precision**, **recall**, **F1-score**, and **ROC-AUC** rather than relying solely on accuracy.

2.2 Data Preprocessing

First, the data needs to be preprocessed before it can be used for modeling. We removed 24 rows with missing values. Then we cleaned the raw review texts to ensure consistency. This included tasks such as removing **line breaks**, **punctuation**, and **special characters**. In addition, the entire text was converted to lowercase. The cleaned text was then tokenized, splitting each review into individual words for evaluation.

2.3 Data Analysis

2.3.1 Descriptive Analysis

After preprocessing, the dataset contained **102,000** reviews with an average of **52.86** words per review and **108,199** unique words throughout the corpus. This indicates the diversity of vocabulary.

The 50 most frequent words turned out to be typical stopwords such as **the**, **and**, **to**, and **is**, but also include some domain-specific terms like **movie**, **great**, **good**, and **bad**, reflecting both sentimental and vocabulary elements. The full list of 50 words can be found in the Appendix (Figure 1).

2.4 Bag-of-Words

To gain insights from domain-specific words further into the analysis, common stopwords such as **the**, **and**, **to**, and **is** were removed using the English stop word list provided by the **scikit-learn** Python library (Pedregosa et al., 2011). After stopword removal, the 100 most common words were extracted to form the **Bag-of-Words** vocabulary. After refining, now the most common words include terms such as **movie**, **great**, **good**, **like**, and **love** making it easier to focus on analysis. The 100 most common words after stopword removal are shown in Table 2

2.4.1 Mutual Information Scores

For further analysis, mutual information scores were calculated between the 100 most common words and the binary target label. The highest-scoring terms include:

- great - 0.011
- bad - 0.0009
- love - 0.0007
- didnt - 0.0006

The 10 terms with the highest mutual information scores are displayed in Figure 3 in the Appendix.

3 Feature Engineering

3.1 Feature Representation

Machine Learning models cannot process textual input, so the textual data was converted into numerical data using two methods, namely **TF-IDF** (Term Frequency-Inverse Document Frequency) and **Word2Vec**. Firstly, TF-IDF vectors were generated using the previously identified 100 most common words after stopwords removal. The main purpose of TF-IDF is to assign higher weights to words that are frequently found in a review but are rare cumulatively across all reviews. This approach highlights discriminative terms. A logistic regression model has been trained on these vectors, resulting in an accuracy of 83%. Secondly, Word2Vec was used to capture semantic relationships between words by mapping each word into a vector space. For this, two Word2Vec models were considered. One being custom-trained on the cleaned Amazon dataset which can capture domain-specific vocabulary and slang tailored to the movie review style. It also gives us more control over the training hyperparameters such as vector size. However, training requires time and resources and may produce lower-quality embeddings for rare words due to limited data size. To compare, we also used a pretrained Word2Vec model: the GloVe 50-dimensional vectors trained on the Wikipedia Gigaword corpus (Pennington et al., 2014), which was loaded using the Gensim Python library (Řehůřek Sojka, 2010). The pretrained model contains rich semantic information from real-world data which does not require any training and provides robust representations but might lack nuance regarding the slang in our data set.

Considering that the created models might be used on reviews on our own e-commerce platform, which are not necessarily movie TV-related, we chose to continue with the pretrained model. It has the advantage of being trained on a much bigger data set than ours, and the domain-specificity of our own model is not something that will be useful for our plans with the model. The Amazon data set is not large enough to train Word2Vec reliably. More advanced embeddings such as those from **BERT** are explored in the **Topic Modeling** section via **BERTopic implementation**

To derive document-level features from the word embeddings, we used **TF-IDF** scores as the weighting schema. This method computes the document vectors as the weighted average of its constituent word vectors, where each word's contribution is scaled by the TF-IDF score. This ensures that notable words contribute proportionally more to the final document representation. This approach enhances representation, while preserving the original dimensionality of the word embeddings. These document embeddings can then be fed as input to classifiers such as **KNN, Naïve Bayes, and Logistic Regression**.

3.2 Feature Selection and Dimensionality Reduction

Word2Vec embeddings are typically high-dimensional, which can be difficult to analyze directly. Therefore, we applied Principal Component Analysis (PCA) and Independent Component Analysis (ICA) to the Word2Vec embeddings to achieve a two-dimensional space, which can be represented visually. PCA offers insights into semantic

relationships while ICA extracts components that are statistically independent. Visualizations of the embeddings show clusters of semantically similar words with positive sentiment terms separated from negative terms. These results validate that the embeddings give meaningful sentiment information emphasizing selected feature representations for sentiment classification.

As seen in Figures 1 and 2, dense clusters appear in the core, with some sentiment-bearing and domain-specific words distinguishable at the edges.

4 Training Procedure

We used the document embeddings (as outlined in section 3.1) to train three different classifiers: Logistic Regression, Gaussian Naïve Bayes, and K-Nearest-Neighbor (KNN).

The Logistic Regression is a standard baseline model. It models the linear relationship between the document vector and the rating of the review. It is easy to train and easy to interpret. Gaussian Naïve Bayes underlies the assumption of feature independence, which is often not met, but is fast to train and often performs well. KNN is a distance-based algorithm that does not make any assumptions about the data and is therefore better at capturing the structure of the data if it is not linear or does not meet the Bayesian assumptions. For Logistic Regression, L2 regularization strength and solver type were tuned via Grid Search to optimize ROC-AUC.

For KNN and the Logistic Regression which was hyperparameter-tuned using L2 regularization, we applied Standard-Scaler from the **scikit-learn** Python library (Pedregosa et al., 2011). This is necessary because both distance-based models and regularized linear models are sensitive to feature scale. Scaling prevents features with larger numerical ranges from skewing the results.

5 Evaluation

We assessed the performance of the three classifiers using Stratified K-fold Cross-Validation. Accuracy, precision, recall, F1, and ROC-AUC were reported, while confusion matrices and classification reports used out-of-fold predictions for fair evaluation. The hyperparameter-tuned Logistic Regression on Word2Vec embeddings achieved the best balance of accuracy, F1, and ROC AUC. Naïve Bayes showed high precision but low recall on dense embeddings, consistent with its limitations for continuous vectors, so it missed many positive reviews. KNN achieved good accuracy and F1 Score, but has a slightly lower ROC-AUC than the Logistic Regression. See Table 4 in the Appendix for accuracy, F1-score and ROC-AUC metrics for all models. With this outcome, we can predict whether a review is positive (≥ 4) or negative (≤ 3). We can automatically flag sentiment on scale, prioritize negative reviews for action.

6 Clustering and Topic Modeling

We used the same Word2Vec document vectors to group reviews (clustering) and to extract common themes (topic modeling). For clustering, we ran two algorithms: Agglomerative (forced into 2 clusters) and DBSCAN (density based, finds dense groups and outliers). For topic-modeling we ran BERTopic on a random sample of reviews to find the most frequent themes and representative keywords.

The two-cluster Agglomerative run tends to split reviews into two broad groups, often aligning roughly with positive vs negative sentiment because the embeddings capture sentiment signals. DBSCAN found several small, dense groups of reviews plus many points labeled as noise; those dense groups usually correspond to repeated, specific complaints or praise (e.g., many short reviews using the same words).

Figures 3 and 4 show visualizations of the agglomerative clustering and DBSCAN, respectively.

BERTopic returned a set of interpretable topics (each shown as a few keywords). These topics represent recurring

customer concerns and praises, for example, comments about actors/plot, sound/visual quality, product packaging, or recommendations. Each topic comes with example reviews so we can validate what it means.

7 Discussion and Limitations

The final logistic regression model depicts high accuracy in predicting the sentiment of a customer review. Before deploying this model, however, a few things need to be considered.

Attention to the preprocessing techniques is important. Siino, Tinnirello, and La Cascia (2024) looked at the impact that different preprocessing methods had on different datasets with different deep models used for sentiment analysis. One of the datasets they analyzed was a IMDb review dataset, which is very similar to our dataset at hand. They found that their Electra and CNN models performed best on data that was only lowercased, but that RoBERTa and XLNet performed best on text which had stopwords removed. They also looked at 3 other datasets, which showed very different results in which preprocessing technique lead to the best results. This suggests that to which extent and how data should be preprocessed, heavily depends on the dataset, but also the model that is going to be used. It would be interesting to test our models with different preprocessing methods and see whether this affects model performance.

The models in this report have been trained on Amazon reviews on movies and TV, which is why they might not perform as well in other domains. If we are to use these models on our own e-commerce platform reviews, it might be necessary to finetune and retrain the models on more domain-specific data. It could also be beneficial to explore alternative Word2Vec embeddings, potentially custom embeddings trained on our own reviews.

The class imbalance was handled by using stratified sampling, but could be further mitigated by techniques such as SMOTE (Synthetic Minority Over-sampling technique). Additionally, vocabulary or sentiment patterns changing could reduce model accuracy over time.

Furthermore, advanced embeddings such as BERT could also be used to train the model, which could improve results by capturing more nuance and context. Fine-tuning a pretrained transformer model on our dataset could substantially enhance performance, particularly for complex expressions of sentiment or sarcasm.

8 References

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Pennington, J., Socher, R., Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543). Association for Computational Linguistics.

Řehůřek, R., Sojka, P. (2010). Software Framework for Topic Modelling with Large Corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks* (pp. 45–50). ELRA.

Siino, M., Tinnirello, I., La Cascia, M. (2024). Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on Transformers and traditional classifiers. *Information Systems*, 121, 102342.

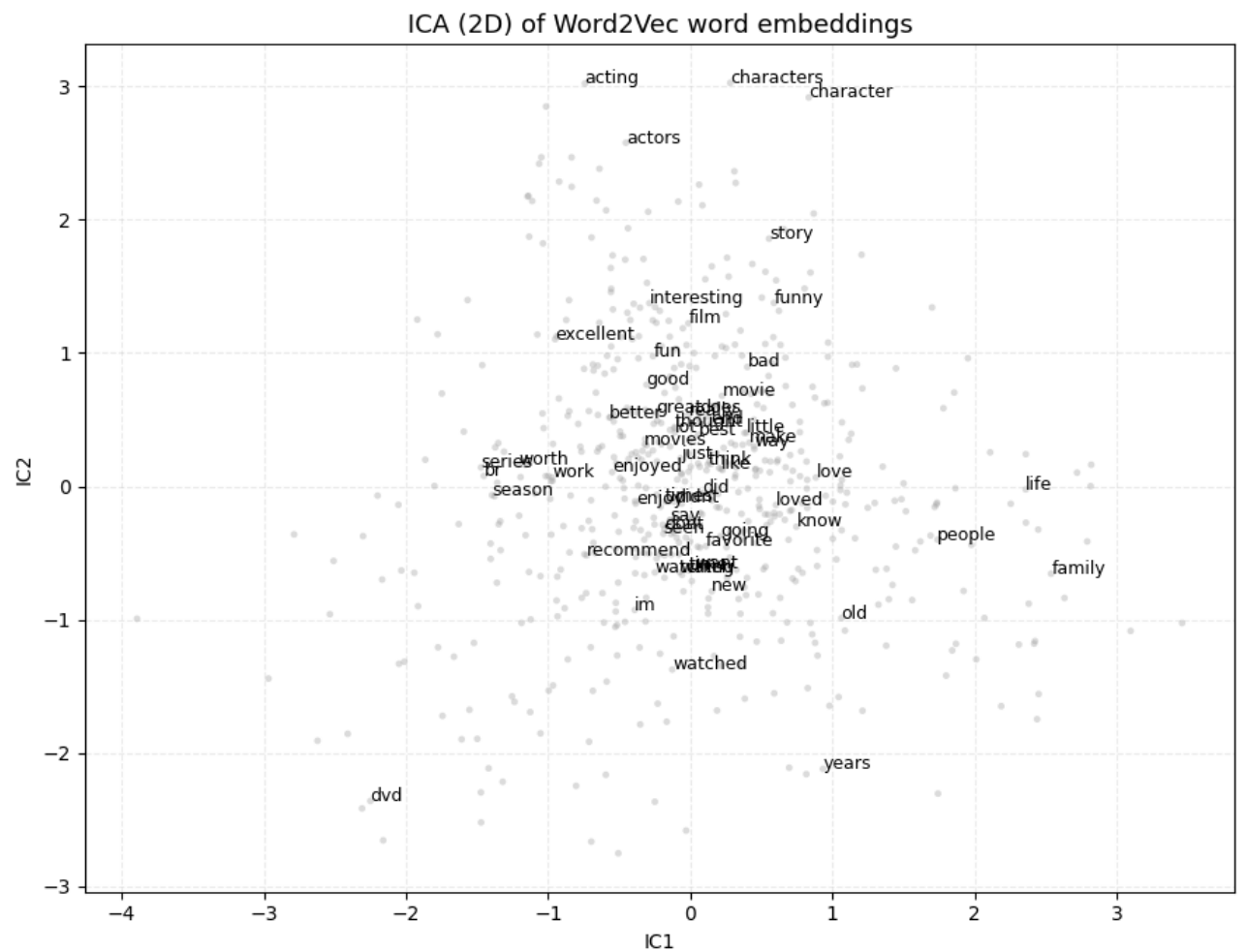


Figure 2: ICA (2D) of Word2Vec word embeddings.

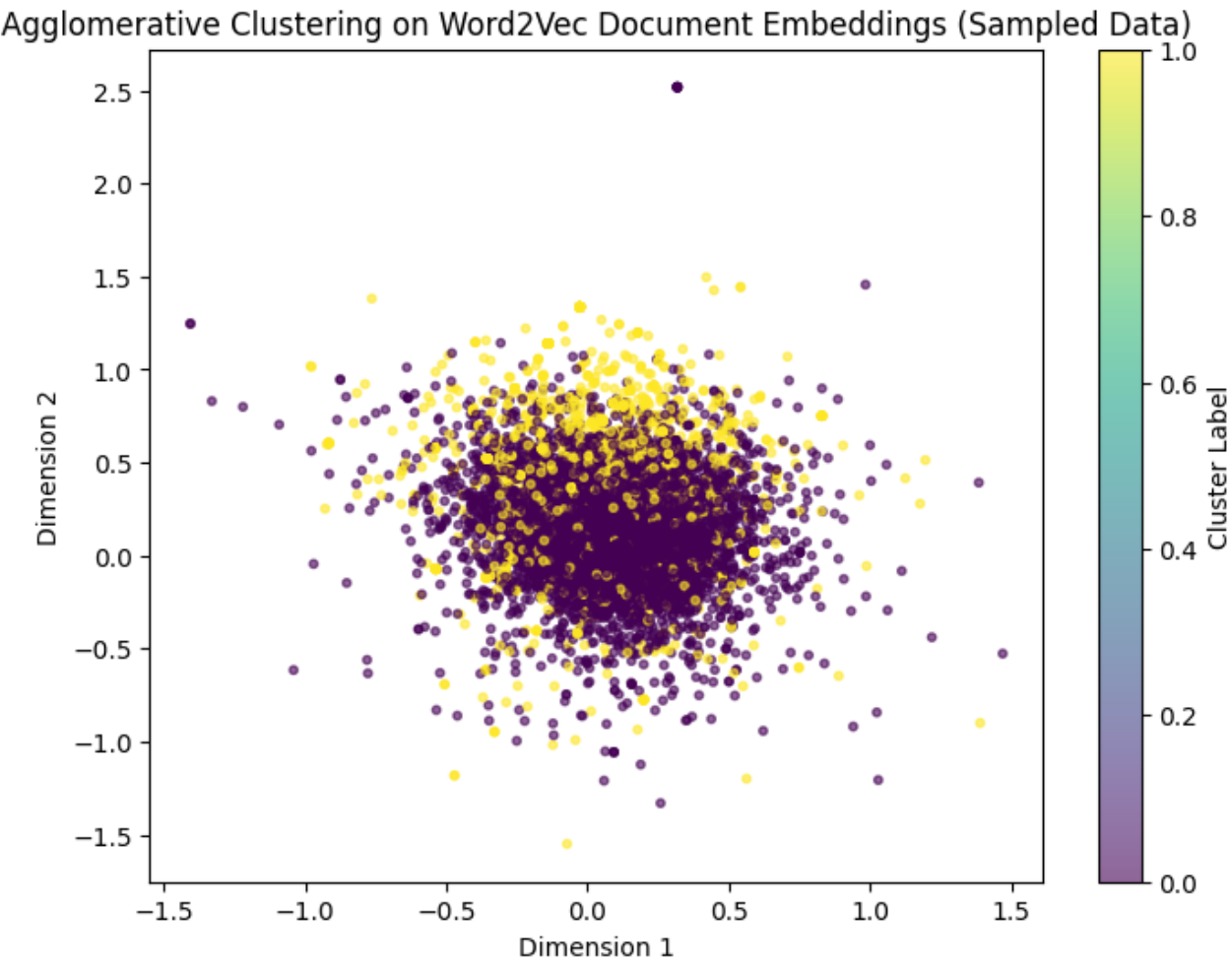


Figure 3: Agglomerative Clustering

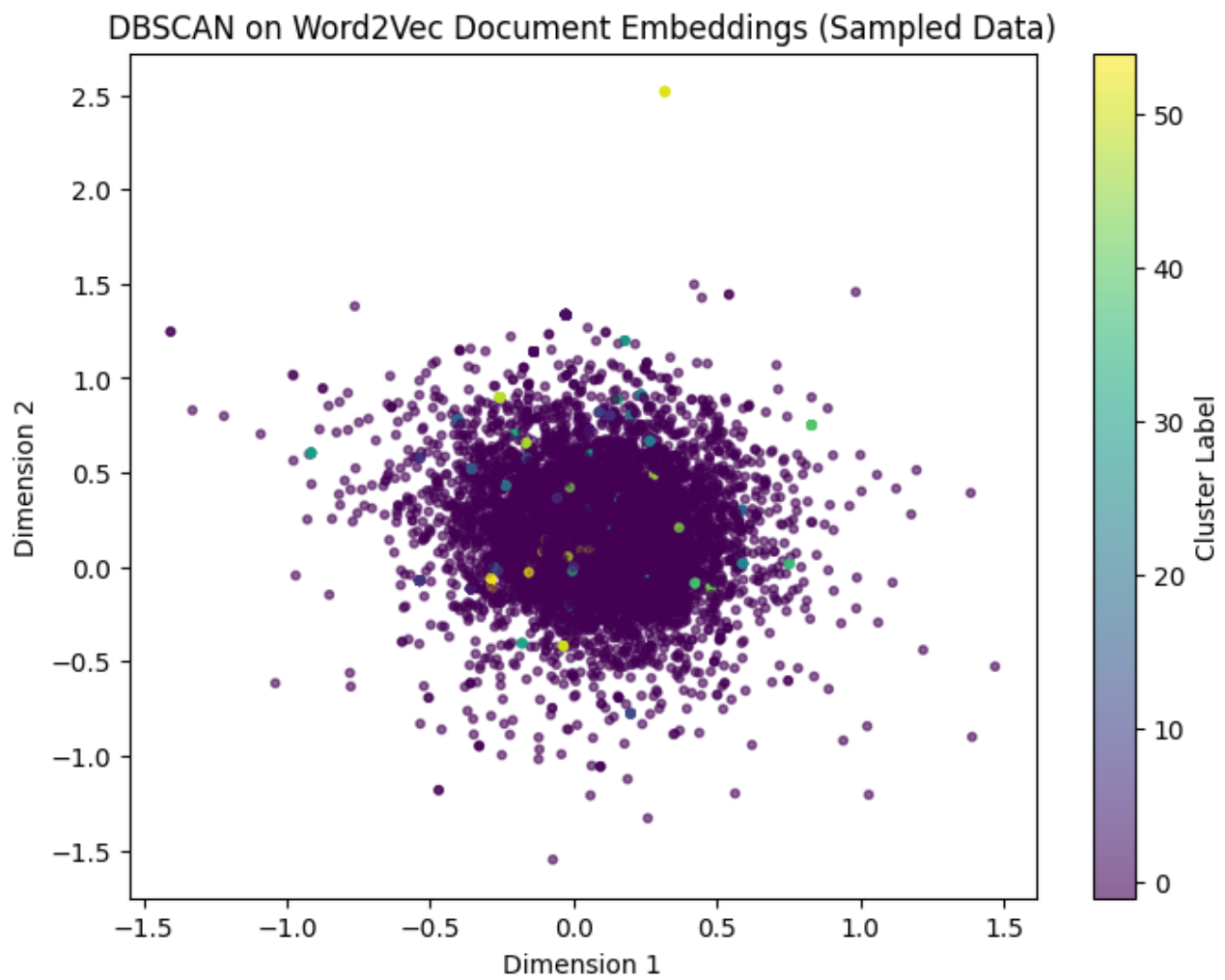


Figure 4: DBScan

Table 1: Top 50 Most Frequent Words in the Corpus

Word	Frequency
the	279,218
and	162,151
a	144,045
to	124,588
of	115,027
i	105,712
is	92,478
this	89,395
it	86,765
in	71,146
for	49,879
that	49,870
was	48,596
movie	47,847
with	43,346
but	38,724
you	38,589
as	37,579
br	35,845
on	34,758
not	31,034
are	30,581
have	27,238
great	24,749
my	24,167
one	23,134
good	22,959
so	22,485
be	22,135
all	21,609
like	19,500
they	19,338
its	19,081
love	18,707
at	17,722
very	17,133
an	16,793
he	16,657
his	16,649
if	16,512
from	16,502
just	16,492
more	16,001
her	14,854
who	14,625
about	14,550
story	14,265
there	14,229
show	14,132
or	14,123

Table 2: List of 100 Words

- movie
- br
- great
- good
- like
- love
- just
- story
- watch
- film
- series
- time
- really
- movies
- dvd
- season
- watching
- dont
- did
- characters
- people
- best
- way
- loved
- little
- think
- im
- better
- acting
- fun
- life
- make
- enjoyed
- watched
- know
- funny
- family
- work
- workout
- new
- didnt
- old
- does
- want
- enjoy
- bad
- seen
- end
- interesting
- 2
- lot
- character
- excellent
- years
- times
- recommend
- worth
- shows
- minutes
- going
- set
- actors
- bit
- video
- ive
- say
- thought
- favorite
- plot
- kids
- real
- music
- feel
- episode
- long
- 3
- got
- makes
- cast
- tv
- different
- things
- action
- episodes
- moves
- liked
- nice
- wonderful
- man
- book
- stars
- 1
- pretty
- fan
- doesnt
- original
- classic
- yoga
- actually
- scenes

Table 3: Top 10 Words by Mutual Information (MI) Score

Word	MI Score
great	0.011071
bad	0.008612
love	0.007429
just	0.006660
didnt	0.006037
minutes	0.004640
loved	0.004477
dont	0.003353
better	0.003249
did	0.003232

Table 4: Model Summary

Model	Accuracy	F1 Score	ROC-AUC
KNN	0.831	0.902	0.786
Naïve Bayes	0.702	0.793	0.759
Logistic Regression	0.826	0.900	0.797